

1:8-Bit ALU Simulation

Studenti:

Borozan Iulia-Adriana

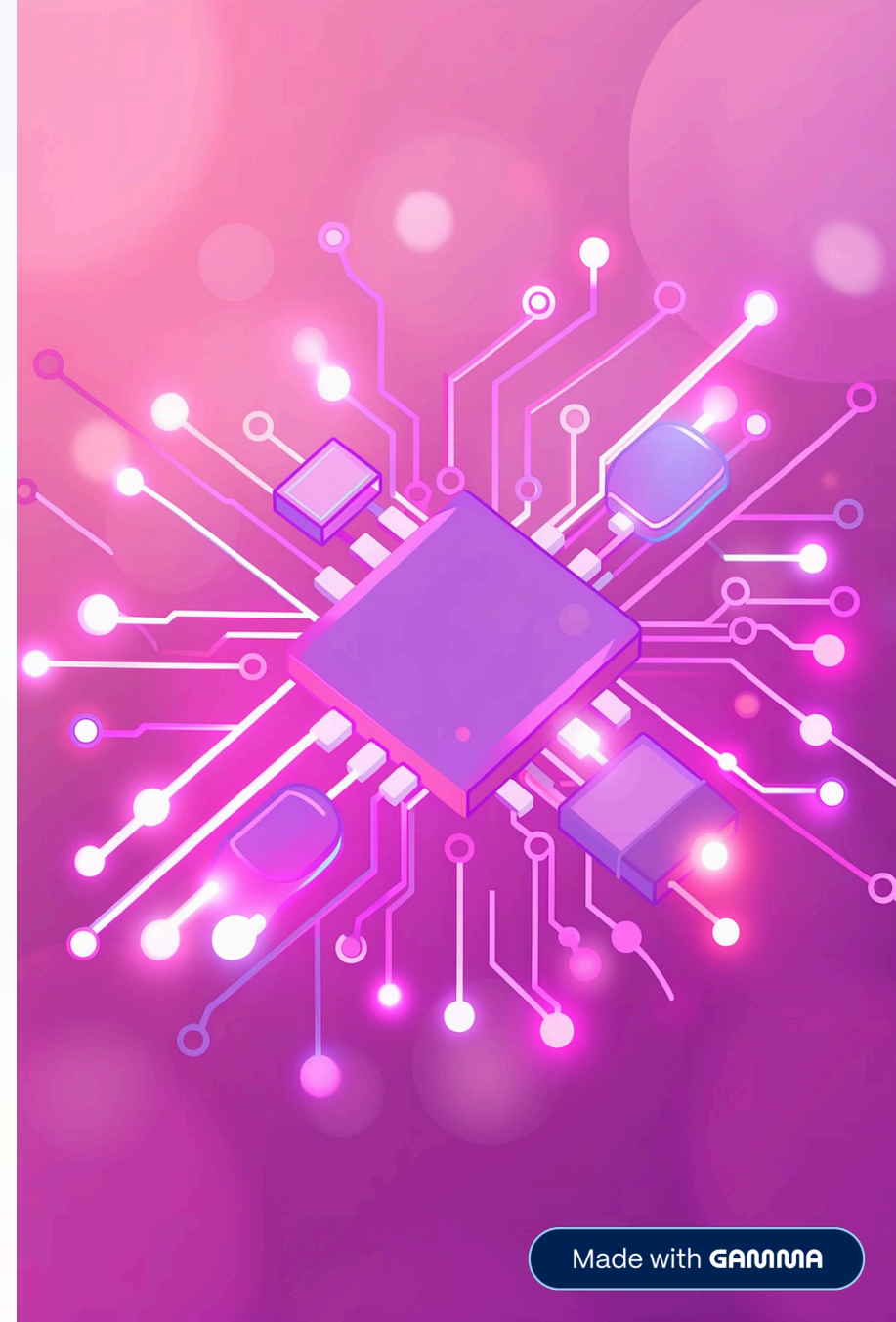
Calotă Denisa-Cosmina

Cîmpan Miriam

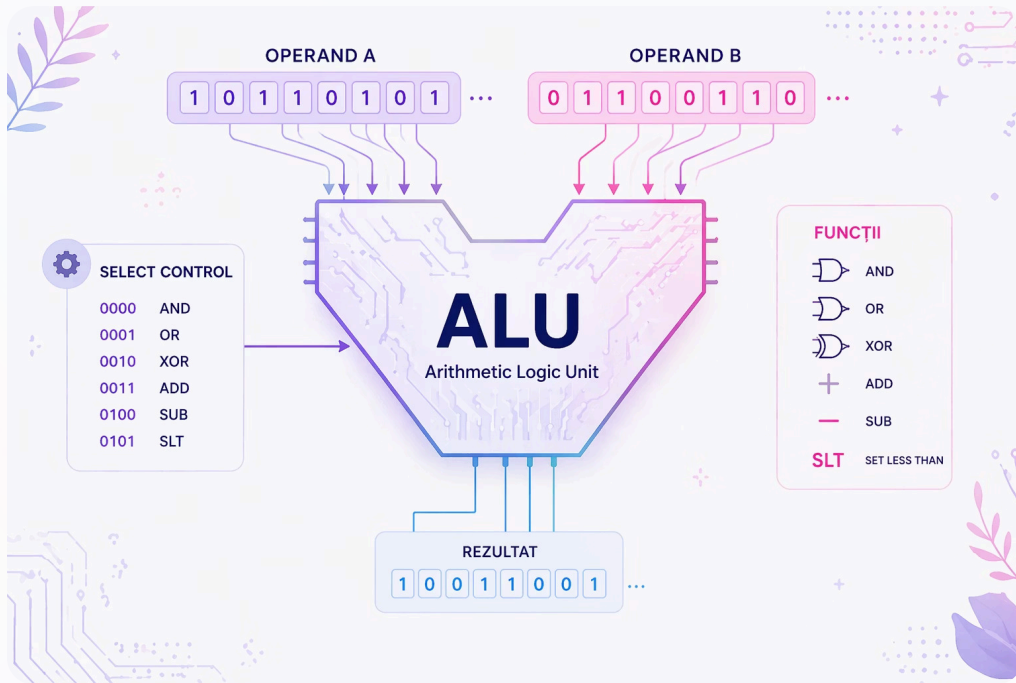
Pîrvănescu Andra Georgiana

Profesor coordonator:

Bozdog Alexandru



Ce este un ALU? Unitatea Aritmetico-Logică



ALU reprezintă blocul funcțional critic al Unității Centrale de Procesare (CPU), fiind un **circuit digital combinațional** proiectat pentru a executa transformări directe asupra operanzilor binari, conform micro-instrucțiunilor primite de la unitatea de control.

Capabilități tehnice

- **Aritmetică în Complement de 2:** Execută adunări, scăderi și operații de shiftare la nivel de bit (LSL/LSR).
- **Logică Booleană:** Implementează funcțiile fundamentale (AND, OR, XOR, NOT) pentru manipularea măștilor de date.

Contextul Execuției (8-Bit Focus)

- **Interfațare:** Preia operanzii direct din **registrele interne** și returnează rezultatul înapoi în datapath într-un singur ciclu de ceas.

Scopul Principal:

Proiectarea și implementarea structurală a unei Unități Logice Aritmetice (ALU) pe 8 biți, utilizând limbajul Verilog HDL.

Operații Implementate: Integrarea a patru operații aritmetice fundamentale, utilizând algoritmi hardware dedicați:

(*) Înmulțire

Utilizând algoritmul Booth Radix-4 (optimizat pentru numere cu semn).

(/) Împărțire

Implementată prin algoritmul Non-Restoring.

(+ -) Adunare și Scădere

Implementate printr-un Ripple Carry Adder (RCA).



Principiul de Design (Arhitectura):



Separarea logicii

Împărțirea clară între Unitatea de Control (automat de stări - FSM) și Calea de Date (Datapath).



Optimizarea resurselor hardware

Toate cele 4 operații împart aceleași resurse fizice (un singur sumator RCA și un set comun de registre), direcționarea datelor făcându-se dinamic prin multiplexoare.



Extensia de bit:

Deși ALU este pe 8 biți, calculele interne folosesc registre pe 9 biți pentru a gestiona corect bitul de semn și depășirile (overflow).

Principiul de Design (Arhitectura):

Arhitectura Modulară

Proiectul este structurat ierarhic pentru a asigura testabilitatea și reutilizarea componentelor.



Modulul Top (**alu.v**)

Unitatea principală care instanțiază și conectează cele două blocuri majore: control_unit și alu_datapath.



Calea de Date (**alu_datapath.v**)

Conține logica de calcul și stocare, fiind formată din:

- **adder_rca.v**: Sumator de tip Ripple Carry pe 9 biți.
- **rgst.v**: Registre universale pentru stocarea operanzilor (A, Q, M).
- **counter.v**: Contor pentru monitorizarea iterațiilor în algoritmii Booth și Non-Restoring.
- **mux_b.v**: Multiplexor pentru selectarea intrării sumatorului.



Unitatea de Control (**control_unit.v**)

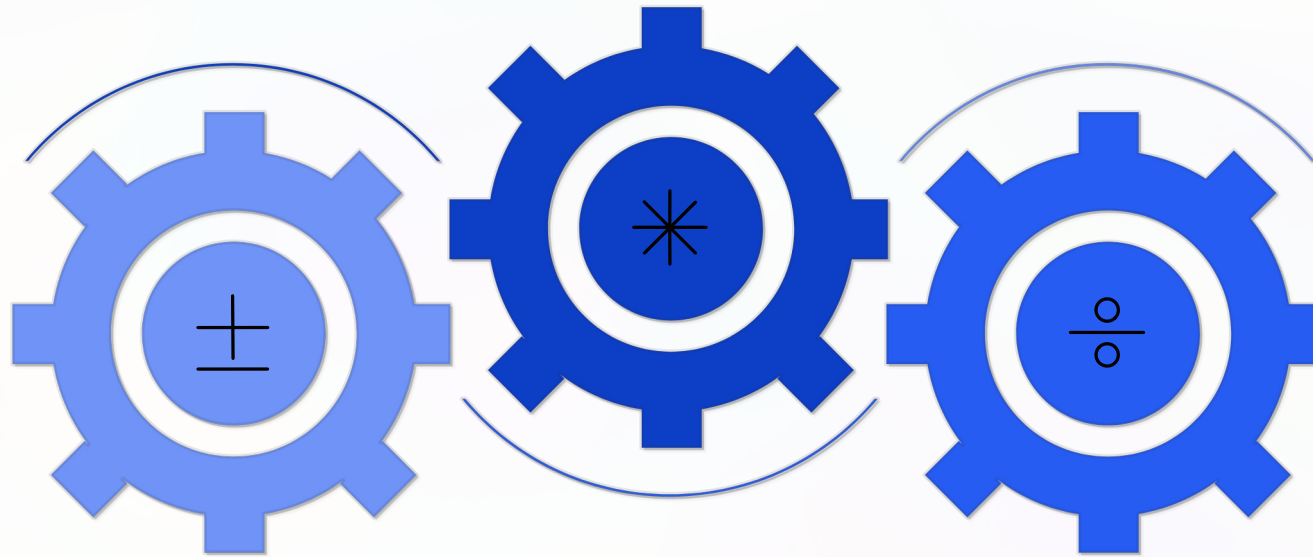
Implementată ca un automat de stări (FSM) care generează semnalele de control (C0-C8) pe baza codului operației (op_code).

Registre si blocuri funcționale

Unitățile de Stocare (Registrele)	Blocuri Funcționale Cheie
Registrul A (Acumulator) Rol critic în toate operațiile; stochează rezultatele parțiale, partea superioară a produsului (Booth) sau restul la diviziune.	Sumatorul RCA (Ripple Carry Adder) Singurul bloc aritmetic principal utilizat; realizează atât adunarea, cât și scăderea (prin activarea semnalului C4 pentru complementul față de 2).
Registrul Q (Multiplicator/Cât) Inițial deține operandul; devine registru de shiftare pentru algoritmul Booth sau stochează câtul rezultat la diviziune.	Multiplexoare de Intrare (MUXES) Selectează dinamic valorile transmise către sumator (ex: M, 2M sau 0) în funcție de necesitățile algoritmului Booth.
Registrul M Stochează multiplicandul sau divizorul, menținând valoarea stabilă pe parcursul iterațiilor.	Logica de Shiftare Integrată direct la intrarea registrelor A și Q pentru a permite deplasări aritmetice la dreapta (ASR) sau logice la stânga (LSL).

Algoritmi Implementați

**Înmulțire:
Algoritmul Booth
Radix-4**



**Adunare și Scădere
(RCA)**

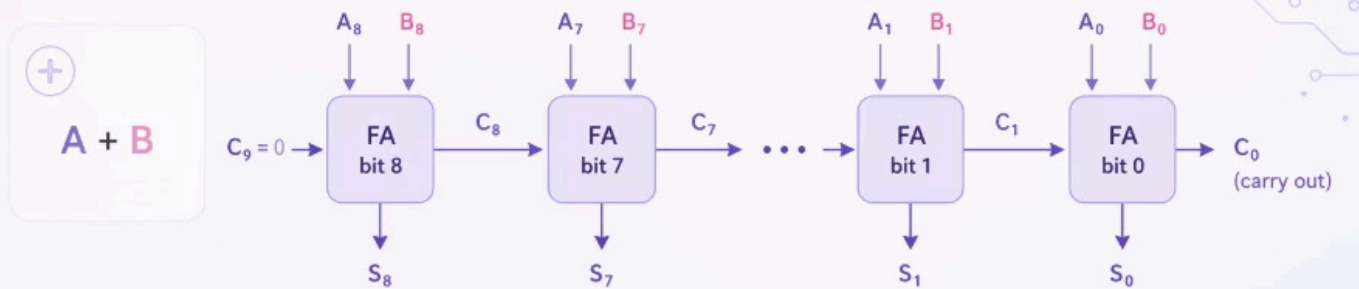
**Împărțire:
Algoritmul Non-
Restoring**

Algoritmi Implementați:

Adunare și Scădere (RCA):

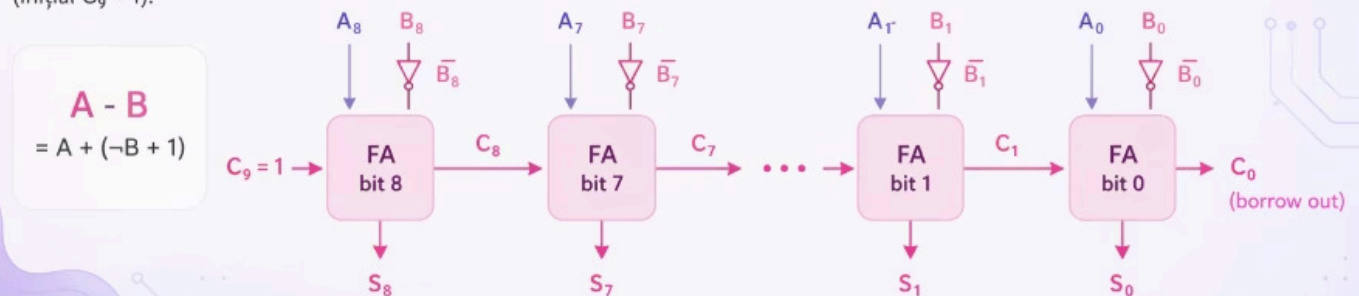
ADUNARE – Ripple Carry Adder (RCA)

Adunarea a două numere pe 9 biți folosind RCA.



SCĂDERE – RCA cu complement față de 2

Scăderea $A - B$ este realizată prin: $A + (\text{complement față de 2 al lui } B)$ (inițial $C_9 = 1$).



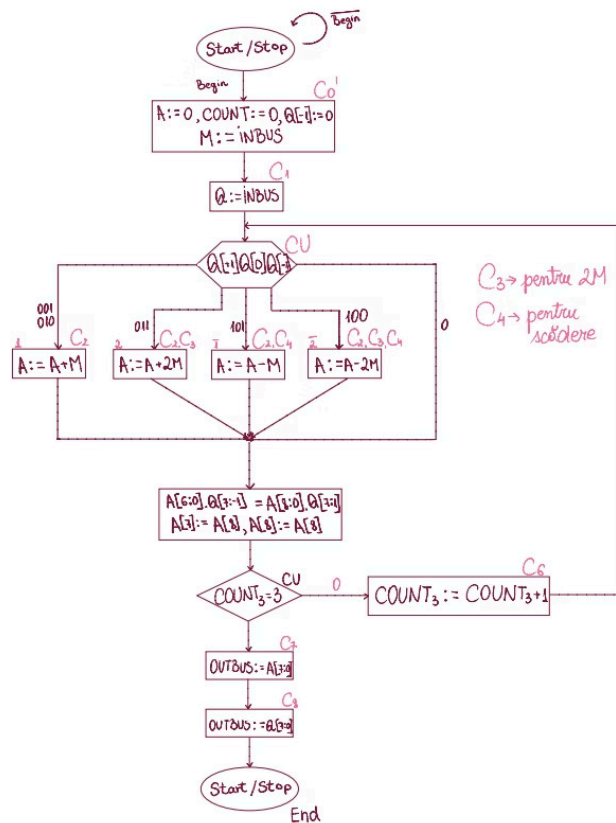
Utilizarea unui **Ripple Carry Adder** pe 9 biți pentru simplitate structurală.

Scăderea este realizată prin adunarea primului operand cu complementul față de 2 al celui de-al doilea (activat de semnalul **C4**).

Algoritmi Implementați:

Înmulțire: Algoritmul Booth Radix-4

Radix - 4 Booth



Avantaj: Reduce numărul de iterații la jumătate față de Booth standard, grupând câte 3 biți (booth_bits) pentru a decide operația.

Funcționare: FSM-ul analizează biții $Q[1]$, $Q[0]$, $Q[-1]$ și comandă sumatorului să adune/scadă M sau $2M$

(generat prin shiftare via **C3**).

Finalizează fiecare pas cu o **deplasare aritmetică la dreapta (ASR)** pe 2 poziții (semnalul **C5**).

ALGORITMUL BOOTH RADIX-4



Reduce numărul de iterații
la jumătate față de Booth standard
(grupe de 3 biți)



Analizează biții $\{Q[i+1], Q[i], Q[i-1]\}$
și decide operația: $+2M$, $+M$, 0 , $-M$, $-2M$
(generată prin shiftare via C3)



Fiecare pas → **deplasare aritmetică**
la dreapta (ASR) pe 2 poziții (C5)

GENERATOR OPERAND

(prin shiftare via C3)

$+2M$ = $M \ll 1$

$+M$ = M

$-M$ = $-M$ (complement față de 2)
prin inversare + 1

$-2M$ = $-(M \ll 1)$

ÎNMULȚIRE: $A \times B$

REGISTRE

A

n biți

Q

n biți

Q_{-1}

1 bit

GRUPARE RADIX-4

$Q[i+1]$

$Q[i]$

$Q[i-1]$

DECODARE / CONTROL

Determină operația

$Q[i+1]$	$Q[i]$	$Q[i-1]$	OPERAȚIE
0	0	0	0
0	0	1	$+M$
0	1	0	$+M$
0	1	1	$+2M$
1	0	0	$-2M$
1	0	1	$-M$
1	1	0	$-M$
1	1	1	0

ADUNĂTOR / SCĂZĂTOR

n+1 biți



A (n biți)

rezultat → A

DEPLASARE ARITMETICĂ LA DREAPTA

(ASR) pe 2 poziții – semnal C5

$A_{n-1} \dots A_0 \quad Q_{n-1} \dots Q_0 \quad Q_{-1} \gg 2$

FLUXUL ALGORITMULUI



1. Inițializare
 $A = 0$, Q = multiplicator,
 $Q_{-1} = 0$, M = multiplicand



2. Grupează 3 biți
 $\{Q[i+1], Q[i], Q[i-1]\}$
și decide operația



3. Adună/Scade
 M , $2M$ sau 0 la A



4. ASR pe 2 poziții
(C5)



5. Repetă pentru
 $n/2$ iterații

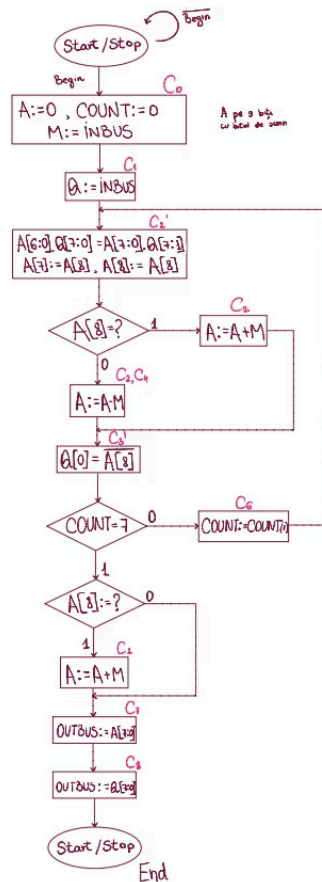


6. Rezultat final
în $[A \ Q]$

Algoritmi Implementați:

Împărțire: Algoritmul Non-Restoring:

Non - Restoring Division



Eficiență: Elimină necesitatea revenirii (restaurării) restului în cazul unei scăderi cu rezultat negativ, accelerând procesul.

Procedură: Începe cu o **deplasare logică la stânga (LSL)** a ansamblului A-Q (semnalul **C2'**).

Decizia de adunare sau scădere a divizorului M se ia în funcție de bitul de semn al lui A (A_sign).

Corecție: La finalul celor 7 pași, se aplică o stare de corecție (**DIV_CORRECT**) dacă restul rămas în A este negativ.

ALGORITMUL NON-RESTORING

Împărțirea lui A la Q (A / Q)



IDEEA DE BAZĂ

Nu „restaurăm” valoarea lui A când scăderea nu este favorabilă — corectăm în pasul următor.



AVANTAJE

- ✓ Mai rapid decât metoda restoring
- ✓ Fără pași de restaurare
- ✓ Corecția se face la final



SEMNE

$A \geq 0 \rightarrow$ pozitiv sau 0
 $A < 0 \rightarrow$ negativ

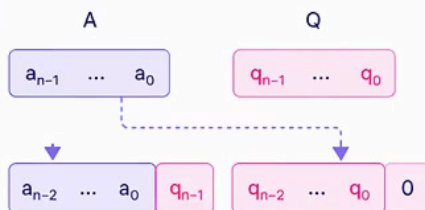


REZULTAT

Câtul este în Q, restul final corectat este în A.

1 SHIFT

Deplasare stânga a (A, Q).



Bitul cel mai semnificativ din Q intră în A.

2 TEST A

Depinde de semnul lui A:

Dacă $A \geq 0$

$A = A - M$

Dacă $A < 0$

$A = A + M$

(M = divizorul)

3 SCRIE Q_0

Setează ultimul bit al lui Q:

Dacă $A \geq 0$

$Q_0 = 1$

Dacă $A < 0$

$Q_0 = 0$

Se construiește astfel câtul (în Q).

4 REPETĂ $\times 8$

Contorul COUNT numără cei 8 pași.



Repetă de 8 ori

CORECȚIE FINALĂ

După cele 8 iterații:

Dacă $A < 0$

$A = A + M$

(restul corectat)

Dacă $A \geq 0$

Nu se modifică A

(restul corectat)

REZULTAT FINAL

Q (câtul)

$q_{n-1} \dots q_0$

n biți

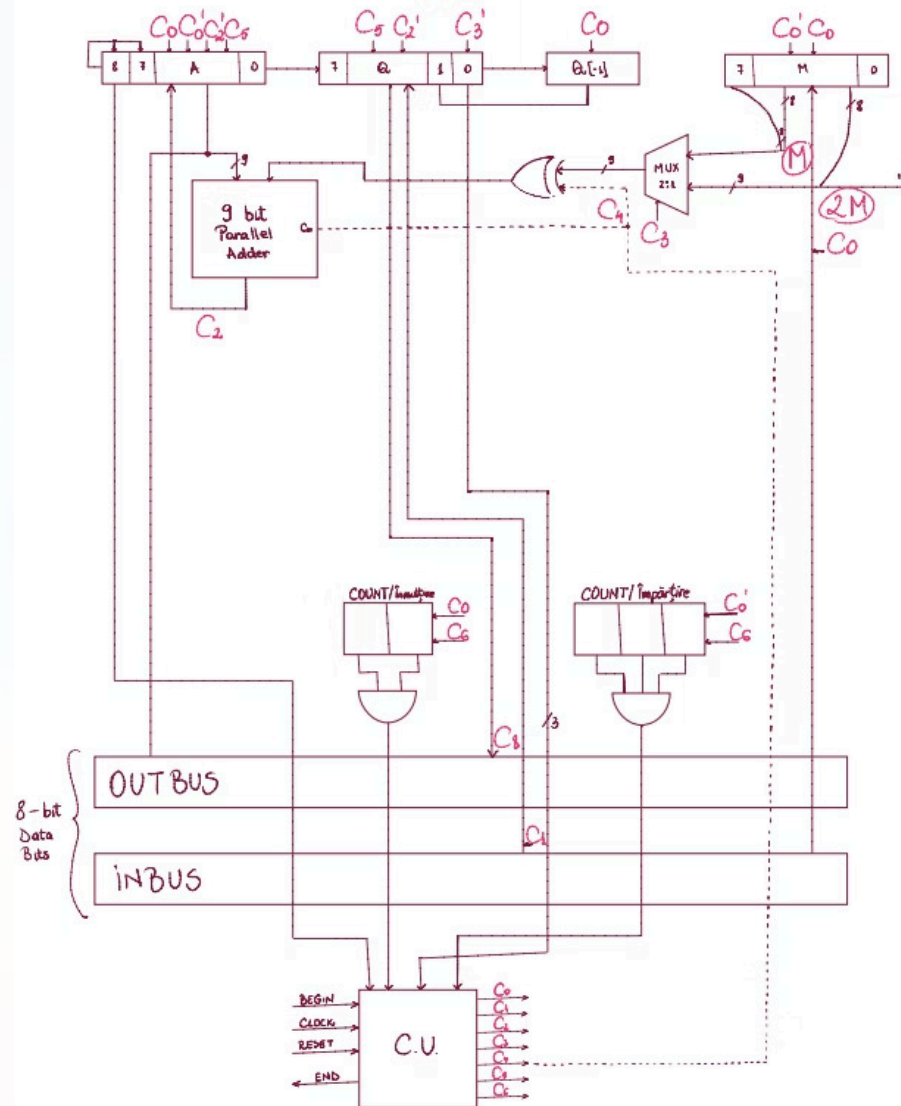
A (restul corectat)

$r_{n-1} \dots r_0$

n biți

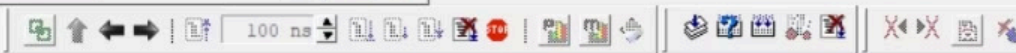


HARDWARE

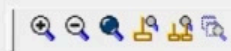




Search:



Layout Simulate



cn/proiect_cn

Name	Status
mux_b.v	✓
alu_tb.v	✓
counter.v	✓
rgst.v	✓
alu_datapath.v	✓
control_unit.v	✓
adder_rca.v	✓
alu.v	✓
fac.v	✓

Objects

Name	Value
clk	1
reset	0
begin_op	0
op_code	00
INBUS	0000..
OUTBUS_A	0001..
OUTBUS_Q	0
end_op	St0

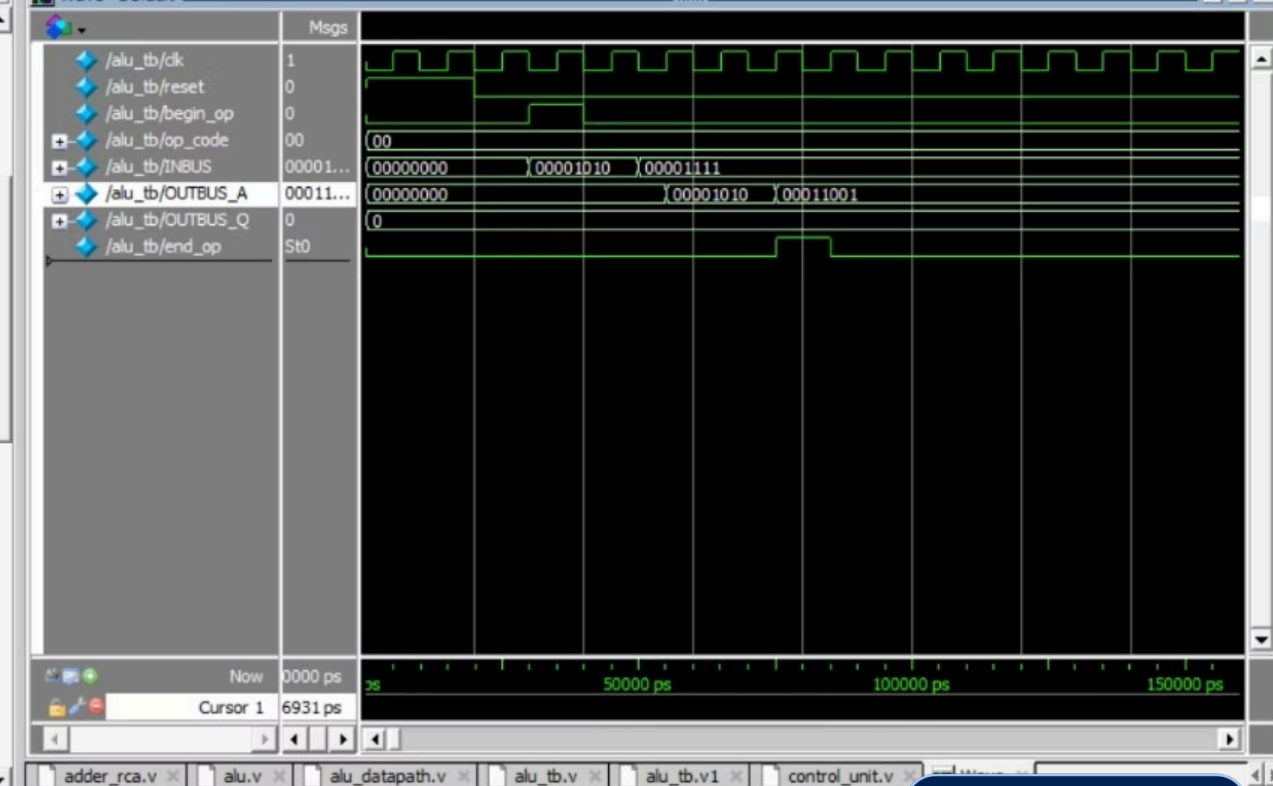
Processes (Active)

Name	Type

Transcript

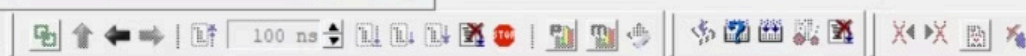
```
# Loading work.control_unit
# Loading work.alu_datapath
# Loading work.mux_b
# Loading work.adder_rca
# Loading work.counter
# Loading work.rgst
# Loading work.fac
add wave -position insertpoint \
sim:/alu_tb/clk \
sim:/alu_tb/reset \
sim:/alu_tb/begin_op \
sim:/alu_tb/op_code \
sim:/alu_tb/INBUS \
sim:/alu_tb/OUTBUS_A \
sim:/alu_tb/OUTBUS_Q \
sim:/alu_tb/end_op
VSIM 3> run -all
#
# =====
# Test Adunare: 10 + 15
# Timp curent: 100000
# Rezultat (OUT_A) = 25
# STATUS: PASS (Adunarea functioneaza!)
# =====
#
# ** Note: $stop : C:/proiect_cn/alu_tb.v(75)
# Time: 100 ns Iteration: 0 Instance: /alu_tb
# Break in Module alu_tb at C:/proiect_cn/alu_tb.v line 75
run -all
# Break key hit
```

Wave - Default

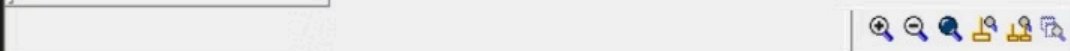




Search:



Layout Simulate



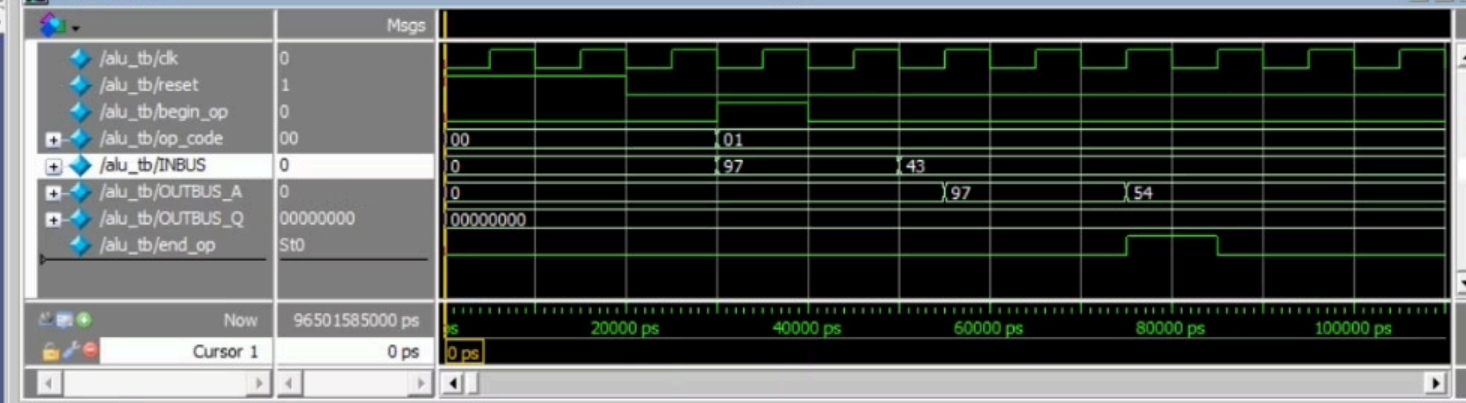
sim - Default

Instance	Design unit	Design
alu_tb	alu_tb	Modul
uut	alu	Modul
#ALWAYS#26	alu_tb	Proce
#INITIAL#29	alu_tb	Proce
#vsim_capacity#		Capac

Objects

Name	Value	Kind	0 ps
clk	0	Regi... Internal	
reset	1	Regi... Internal	
begin_op	0	Regi... Internal	
op_code	00	Pack... Internal	
INBUS	0	Pack... Internal	
OUTBUS_A	0	Net Internal	
OUTBUS_Q	0000...	Net Internal	
end_op	St0	Net Internal	

Wave - Default



Library Project sim

Transcript

```

sim:/alu_tb/end_op
VSIM 3> run -all
#
# =====
# Test Scadere: 97 - 43
# Timp curent: 100000
# Rezultat (OUT_A) = 54
# STATUS: PASS (Scaderea functioneaza!)
# =====
#
# ** Note: $stop : C:/proiect_cn/alu_tb.v(75)
# Time: 100 ns Iteration: 0 Instance: /alu_tb
# Break in Module alu tb at C:/proiect_cn/alu tb.v line 75

```

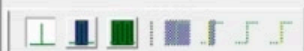
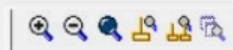
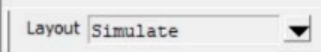
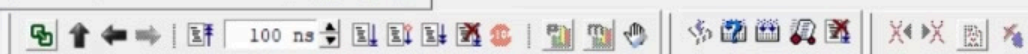
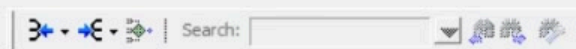
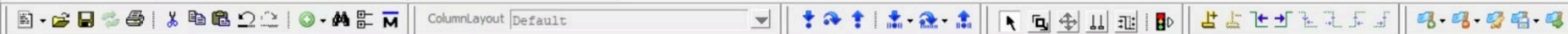
Instance	Design unit	Desi
alu_tb	alu_tb	Mod
uut	alu	Mod
#ALWAYS#26	alu_tb	Proc
#INITIAL#29	alu_tb	Proc
#vsim_capacity#		Cap

Name	Value	Kind	Now
clk	-1	Regi... Internal	
reset	0	Regi... Internal	
begin_op	0	Regi... Internal	
op_code	-2	Pack... Internal	
INBUS	11	Pack... Internal	
OUTBUS_A	-2	Net Internal	
OUTBUS_Q	28	Net Internal	

Name	Type (filtered)	State	Order
#INITIAL#29	Initial	Active	1
#ALWAYS#26	Always	Ready	2



```
sim:/alu_tb/end_op
VSIM 3> run -all
#
# =====
# Test Inmultire: 11 * (-44)
# Timp curent: 350000
# Rezultat pe 16 biti = -484
# STATUS: PASS (Inmultirea functioneaza)
# =====
#
# ** Note: $stop : C:/proiect_cn/alu_tb.v(79)
# Time: 350 ns Iteration: 0 Instance: /alu_tb
```



Default

Instance

alu_tb

uut

#ALWAYS#26

#INITIAL#29

#vsim_capacity#

OUTBUS_A

OUTBUS_Q

Name

#INITIAL#29

#ALWAYS#26

project

sim

Wave

alu_tb.v

Transcript

#

Test Impartire: 115 / 10

Timp curent: 450000

Catul (OUT_Q) = 11 | Restul (OUT_A) = 5

STATUS: PASS (Impartirea functioneaza)

#

** Note: \$stop : C:/proiect_cn/alu_tb.v(79)

Time: 450 ns Iteration: 0 Instance: /alu_tb

Break in Module alu_tb at C:/proiect_cn/alu_tb.v line 79

Wave - Default

Msgs

/alu_tb/clk 1

/alu_tb/reset 0

/alu_tb/begin_op 0

/alu_tb/op_code 3

/alu_tb/INBUS 115

/alu_tb/OUTBUS_A 5

/alu_tb/OUTBUS_Q 11

/alu_tb/end_op 0

Now 450000 ps

Cursor 1 0 ps

50000 ps 100000 ps 150000 ps 200000 ps 250000 ps 300000 ps 350000 ps

project

sim

Wave

alu_tb.v

Transcript

#

Test Impartire: 115 / 10

Timp curent: 450000

Catul (OUT_Q) = 11 | Restul (OUT_A) = 5

STATUS: PASS (Impartirea functioneaza)

#

** Note: \$stop : C:/proiect_cn/alu_tb.v(79)

Time: 450 ns Iteration: 0 Instance: /alu_tb

Break in Module alu_tb at C:/proiect_cn/alu_tb.v line 79



Mulțumim pentru atenție! 🎀